Fichas bibliográficas - FORM-FB-01 Fundamentos de Sistemas Distribuidos y Arquitecturas Distribuidas

Steffanny del Rocío Loor Suárez

07 de noviembre de 2026

Ficha 1: Consistencia causal

Resumen

En este artículo los autores abordan el problema de mantener la consistencia causal de los datos entre servidores distribuidos geográficamente, evitando anomalías en las lecturas sin afectar demasiado el rendimiento. Para ello, Freitas et al. [1] realizaron una revisión de la literatura científica, donde examinaron y compararon sistemas que implementan consistencia causal y admiten transacciones de solo lectura. Como resultado, identificaron cuatro estrategias recurrentes: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas. Además, analizaron sus ventajas y limitaciones, destacando que cada estrategia implica compromisos relacionados con la escalabilidad, latencia, coordinación y rendimiento de los sistemas distribuidos.

Dato concreto

Los autores identifican cuatro estrategias recurrentes para implementar consistencia causal: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas.

Ubicación: Página 10, sección 3.5 (Summary), tabla 1: Recurrent Strategies to Implement Causally Consistent Systems [1].

Nota personal

Este artículo me ayudó a comprender que es difícil mantener un equilibrio entre consistencia, disponibilidad y rendimiento en los sistemas distribuidos. Me pareció interesante la consistencia causal de la que se habla en el artículo, así como también las cuatro estrategias identificadas por Freitas et al. [1]: comprobación de dependencias, secuenciadores, protocolos de estabilización y enfoques optimistas, ya que cada una presenta ventajas y limitaciones. En el informe TA-IND-01 utilizaría esta información para analizar cómo un

sistema distribuido mantiene la consistencia de sus datos y qué compromisos existen entre las diferentes alternativas, relacionándolos con los conceptos de CAP y PACELC.

Ficha 2: Tolerancia a fallos y recuperación de nodos

Resumen

Los fallos permanentes de nodos en clústeres de procesamiento pueden ocasionar problemas como interrumpir la ejecución de tareas distribuidas. Para enfrentar esta situación, Reitz y Fohry [2] proponen un mecanismo de puntos de control a nivel de tarea que permite recuperar el trabajo afectado sin reiniciar toda la aplicación. Esta propuesta fue implementada y evaluada mediante cuatro programas de prueba en dos clústeres y los resultados demostraron que sí es posible continuar con la ejecución utilizando los recursos disponibles, aunque mantener esta protección genera una sobrecarga adicional en el tiempo de procesamiento.

Dato concreto

En los experimentos realizados con 1536 trabajadores en el clúster Lichtenberg, Reitz y Fohry registraron una sobrecarga máxima de protección del 43,98 %, mientras que se obtuvo el 28,3 % con 1280 trabajadores en el clúster Goethe. Estos resultados muestran que los programas necesitaron más tiempo para ejecutarse al utilizar el mecanismo de protección mediante puntos de control, en comparación con las ejecuciones sin protección [2].

Ubicación: Página 8 del PDF, apartado *Experiments*, subsección *Failure-Free Runs*, párrafo que comienza con “The results from Fig. 5 are summarized in Fig. 6” [2].

Nota personal

Aprendí un poco más sobre los mecanismos que permiten recuperar las tareas afectadas sin tener que reiniciar toda la aplicación cuando falla un nodo en un sistema distribuido. Utilizar los puntos de control puede aumentar el tiempo de procesamiento, pero ayuda a mantener la continuidad de las tareas ante determinados fallos [2]. En el informe TA-IND-01 podría utilizar esta información para analizar cómo un sistema distribuido puede recuperarse de los fallos y qué consecuencias tienen estos mecanismos en su rendimiento.

Ficha 3: Evolución de las API en microservicios

Resumen

Cuando se modifican las API de los microservicios pueden surgir problemas, ya que estos cambios pueden afectar la compatibilidad y la comunicación entre los servicios. Para investigar esta situación, Lercher et al. [3] realizaron entrevistas semiestructuradas a 17

profesionales de 11 empresas, con el propósito de conocer sus experiencias y las estrategias que utilizan. A partir de los resultados, identificaron diferentes estrategias y desafíos relacionados con la evolución de las API. Entre ellos destacan la necesidad de mantener la compatibilidad con versiones anteriores, coordinar los cambios entre equipos y mejorar la comunicación para evitar problemas de integración entre los diferentes servicios involucrados.

Dato concreto

En la investigación de Lercher et al., los 17 participantes señalaron que, en determinadas situaciones, es necesario realizar cambios en las API que pueden romper la compatibilidad con versiones anteriores. Esto indica que, aunque mantener la compatibilidad es importante, algunos cambios son necesarios para incorporar nuevas funcionalidades o mejorar los servicios [3].

Ubicación: Página 6 del PDF, apartado *4. API evolution strategies (RQ2)*, subsección *4.2. Accept necessary breaking changes*, primer párrafo [3].

Nota personal

Con este artículo comprendí que modificar una API no solo afecta al servicio donde se realiza el cambio, sino que también puede ocasionar problemas en otros servicios importantes que dependen de ella. También aprendí que mantener la compatibilidad es importante, aunque en algunas situaciones sea necesario introducir cambios que la rompan [3]. Para el informe TA-IND-01, esta información me ayudará para analizar de mejor manera cómo se comunican los componentes de un sistema distribuido y qué dificultades pueden surgir cuando se modifican sus interfaces.

Ficha 4: Planificación y distribución de microservicios

Resumen

El principal problema que aborda Alelyani et al. [4] es que la comunicación entre microservicios distribuidos puede generar demasiado tráfico, retrasos y un uso ineficiente de los recursos. Para solucionarlo, los investigadores propusieron una estrategia de planificación que combina un algoritmo de optimización por enjambre de partículas (PSO) modificado y Round Robin. El primero selecciona servidores apropiados para colocar réplicas de microservicios, mientras que el segundo las distribuye equilibradamente. La estrategia integra Kubernetes para gestionar su despliegue. Mediante simulaciones con datos de Alibaba y Google, comprobaron mejoras en el tráfico de red, el balanceo de carga y el aprovechamiento de recursos.

Dato concreto

Alelyani et al., en su primer escenario de simulación, consiguieron reducir el tráfico de red en un 36 % con respecto a la estrategia utilizada por Alibaba. Para lograrlo, distribuyeron 11 004 réplicas cerca de los microservicios con los que necesitaban comunicarse [4].

Ubicación: Página 14 del PDF, página impresa 35148, sección *VIII. Results and Discussion*, subsección *A. Reducing Network Traffic Cost*, primer párrafo [4].

Nota personal

En este artículo aprendí que, para distribuir microservicios, es importante analizar dónde se colocan sus réplicas y cómo se comunican entre los servidores [4]. En el informe TA-IND-01, utilizaría este aprendizaje para analizar cómo un sistema distribuido organiza sus componentes y distribuye la carga, con el propósito de evitar retrasos y aprovechar mejor sus recursos.

Referencias bibliográficas

[1] D. Freitas, P. deGrandis y T. B. Sousa, «A Survey on the State of the Art of Causally Consistent Cloud Systems,» *ACM Computing Surveys*, vol. 57, n.º 9, mayo de 2025, art. núm. 240. DOI: 10.1145/3731444 dirección: https://doi.org/10.1145/3731444

[2] M. Reitz y C. Fohry, «Task-Level Checkpointing and Localized Recovery to Tolerate Permanent Node Failures for Nested Fork–Join Programs in Clusters,» *SN Computer Science*, vol. 5, n.º 3, 2024, art. núm. 320. DOI: 10.1007/s42979-024-02624-8 dirección: https://doi.org/10.1007/s42979-024-02624-8

[3] A. Lercher, J. Glock, C. Macho y M. Pinzger, «Microservice API Evolution in Practice: A Study on Strategies and Challenges,» *Journal of Systems and Software*, vol. 215, 2024, art. núm. 112110. DOI: 10.1016/j.jss.2024.112110 dirección: https://doi.org/10.1016/j.jss.2024.112110

[4] A. Alelyani, A. Datta y G. M. Hassan, «Optimizing Cloud Performance: A Microservice Scheduling Strategy for Enhanced Fault-Tolerance, Reduced Network Traffic, and Lower Latency,» *IEEE Access*, vol. 12, págs. 35 135-35 153, 2024. DOI: 10.1109/ACCESS.2024.3373316 dirección: https://doi.org/10.1109/ACCESS.2024.3373316